

Department of Computer and Software Engineering

SE: Machine Learning

Course Instructor: Dr. Ahmed Raza

Date: 12-February-2026

Day: Thursday

Semester: 6th

Lab 1: The Engine of Learning: Gradient Descent and KNN

Lab Title	CLO	BT	PLO	Weightage
The Engine of Learning: Gradient Descent and KNN				

Name	Roll Number	Report Marks/10	Viva Marks/5	Total Marks/15
Zunaira Abdul Aziz	BSSE23058			

Checked on: _____

Signature: _____

1.1 Learning Outcomes

After completing this Lab, the student will be able to:

- Implement gradient descent for scalar and vector-valued functions
- Understand convergence, divergence, and learning rate effects
- Implement K-Nearest Neighbors using distance geometry
- Compare loop-based vs vectorized implementations

1.2 Equipment

- PC with Python 3.x installed (Anaconda Distribution recommended).
- IDE of choice (Jupyter Notebook, Spyder, VS Code, or PyCharm).
- NumPy, Pandas, and Matplotlib libraries installed.

1.3 Theory & Background

Machine Learning heavily relies on optimization algorithms and geometric reasoning. In this lab, you will build machine learning algorithms (Gradient Descent and K-Nearest Neighbors) from scratch using math and NumPy. No sklearn. Only optimization, geometry, and engineering discipline.

1.3.1 Gradient Descent

Gradient Descent is an iterative optimization algorithm used to minimize a function by moving in the direction of steepest descent. The update rule is:

$$\theta_{n+1} = \theta_n - \alpha \nabla f(\theta_n)$$

where α is the learning rate and ∇f represents the gradient. The learning rate controls the step size and critically affects convergence behavior.

1.3.2 K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based learning algorithm. It classifies a data point based on the majority class among its k nearest neighbors in the feature space. The algorithm relies on distance metrics, typically Euclidean distance:

$$d(x, y) = \sqrt{\sum_{i=1}^N (x_i - y_i)^2}$$

KNN has no explicit training phase; all computation happens at prediction time, making it memory-intensive for large datasets.

1.4 Submission Format

- Submit the provided filled in python script (.py), along with a manual in PDF format in a zip file.
- Naming convention: BSSE23XXX_SECTION#_ML_LABY (e.g., BSSE23001_A_ML_LAB1), where XXX is the remaining roll number and Y is the lab number.

PART A: Gradient Descent

Task A1: Scalar Gradient Descent [2 Marks]

Function: $f(x) = x^2 + 5$

1. Derive $f'(x)$ analytically.
2. Implement gradient descent starting from $x_0 = 10$.
3. Run for 20 iterations with learning rates: $\alpha = 0.1, 1.0, 1.1$
4. Identify which values converge or diverge and explain why.

Task A2: Gradient Descent on a 2D Surface [2 Marks]

Function: $f(x, y) = x^2 + 3y^2$

1. Derive the gradient vector $\nabla f = [\partial f / \partial x, \partial f / \partial y]$.
2. Start from $(x_0, y_0) = (5, 5)$ with learning rate $\alpha = 0.1$.
3. Track parameter updates for at least 10 iterations.
4. Explain why convergence rate differs for x and y dimensions.

Task A3: Failure Mode [1 Mark]

1. Choose a learning rate that causes divergence.
2. Log the exploding values over iterations.
3. Explain divergence mathematically: why does the value grow instead of decrease?

Task A1:

1. In code file
2. In code file
3. In code file
4. -Alpha 0.1: Final $x = 0.1441$: This works because the step size is small. Each update reduces the value of x until it gets close to the minimum at zero. (Converges)
-Alpha 1.0: Final $x = -10.0000$: This fails because the step is too large. The update formula $(x - (1.0 * 2x))$ simply flips x from 10 to -10 and back, never reaching the bottom. (Oscillates)
-Alpha 1.1: Final $x = -319.4800$: The step size overshoots the minimum so drastically that x grows larger in every iteration, eventually exploding toward infinity. (Diverges)

Task A2:

1. $\nabla f = [\partial f / \partial x, \partial f / \partial y]$

$$\partial f / \partial x = 2x$$

$$\partial f / \partial y = 6y$$

$$F = [2x, 6y]$$

2. Code file

3. Code file
4. The convergence rates for the x and y dimensions differ because the function f is significantly steeper in the y direction. With a learning rate of $\alpha = 0.1$, the x dimension (gradient $2x$) reduces by only 20% per step, while the y dimension (gradient $6y$) reduces by 60% per step. Your tracking data clearly shows this imbalance: while x starts at 5.0 and is still at 0.5369 after 10 iterations, y drops from 5.0 to a near-zero value of 0.0005 in the same timeframe. Because the y updates take much larger percentage leaps toward the minimum, that dimension converges much faster than the x dimension.

Task A3:

1. For the function $f(x) = x^2 + 5$ with a starting point of $x = 10$, any learning rate > 1.0 will cause divergence. We will use $a = 1.1$ to demonstrate this failure mode
2. When the learning rate is set to $a = 1.1$, the algorithm fails because the step size is too large, causing values to explode in magnitude while flipping signs at every step. Starting at $x = 10.00$, subsequent values like -12.00 and 14.40 show that the model overshoots the minimum so drastically that it diverges toward infinity instead of converging to zero.
3. Divergence occurs because the update rule $x_{\text{new}} = x(1 - 2a)$ simplifies to $x_{\text{new}} = -1.2x$ when $a = 1.1$. Since the absolute value of this multiplier (1.2) is greater than 1, the magnitude of x increases by 20% every iteration. This massive step size causes the model to land further away from the center on the opposite side each time, leading to a mathematical explosion of values.

PART B: K-Nearest Neighbors (KNN)

Dataset: A synthetic 2D binary classification dataset generated using NumPy (please refer to function `generate_dataset` in the code file provided).

The dataset is linearly inseparable and reproducible via a fixed random seed.

Task B1: Euclidean Distance [1 Mark]

1. Implement Euclidean distance between two points using loops.
2. Implement Euclidean distance using NumPy vectorization (no loops).
3. Verify both implementations produce the same result.
4. Compare code clarity and efficiency.

Task B2: Distance Computation with Broadcasting [2 Marks]

1. Compute distances from a single test point to all training points.

2. No loops allowed, use NumPy broadcasting.
3. Explain the broadcasting expression in comments.

Task B3: Neighbor Selection (No Full Sorting) [1 Mark]

1. Use `np.argpartition` to find k nearest neighbors without full sorting.
2. Explain why full sorting with `np.argsort` is inefficient for large datasets.
3. Compare time complexity: $O(n \log n)$ vs $O(n)$.

Task B4: KNN Prediction [2 Marks]

1. Retrieve the labels of the k nearest neighbors.
2. Perform majority voting to determine the predicted class.
3. Return the predicted class label.

Task B5 — Effect of k [1 Mark]

1. Test your KNN implementation with $k = 1$, $k = 5$, and $k = n$ (total training points).
2. Identify which values lead to overfitting and which lead to under fitting.
3. Explain the behavior observed.

Task B1:

1. Code file
2. Code file
3. Code file
4. Vectorized code is much easier to read because it uses a single line that looks exactly like the math formula. Loops are noisy and require more lines of code to do the same task. NumPy is incredibly fast because it runs on optimized C-code under the hood. While a Python loop checks every number one by one, vectorization processes entire blocks of data at once, often making it over 100x faster as data grows

Task B2:

1. Code File
2. Code File
3. Code File

Task B3:

1. Code file

2. Full sorting with `np.argsort` is inefficient because it wastes computational power organizing the entire dataset. In KNN, we only care about the k closest neighbors, so there is no need to determine the relative order of the remaining distant points. Using a partial sort avoids this unnecessary work, making it much faster when k is small compared to the total number of points N .
3. $O(n)$ is significantly faster because it only finds the specific k neighbors needed, whereas $O(n \log n)$ wastes time sorting the entire dataset.

Task B4:

1. Code File
2. Code File
3. Code File

Task B5:

1. $k=1$ uses the single nearest neighbor.
 $k=5$ uses a small group consensus.
 $k=100$ (n) considers the entire dataset for every prediction.
2. $k=1$ leads to Overfitting, while $k=n$ leads to Underfitting.
3. At $k=1$ (Overfitting): The model is too sensitive to noise. It creates a wiggly and complex boundary that follows every individual point too closely, failing to generalize to new data. At $k=5$ (Balanced): This provides the best result. It is smooth enough to ignore local outliers while still being flexible enough to capture the true shape of the data. At $k=n$ (Underfitting): The model is too simple. It ignores all local patterns and effectively predicts the same majority class for every point, regardless of its position.

PART C — Reflection [3 Marks]

1. Compare Gradient Descent vs KNN in terms of:
 - Training cost
 - Prediction cost
 - Memory usage
2. Which algorithm scales better to large datasets? Justify your answer.
3. Why is KNN rarely used in production ML systems despite its simplicity?

1. Gradient Descent is used in parametric models that have a high training cost because they must iteratively find optimal weights, but they offer a very low prediction cost and minimal memory usage since only those weights need to be stored. In contrast, KNN is a lazy learner with zero training cost, but it suffers from high memory usage and a heavy prediction cost because it must store and search the entire dataset to calculate distances for every new input.
2. Gradient Descent scales much better to large datasets because the model size remains constant regardless of how much data was used for training. Once the weights are learned, the original data can be discarded. KNN scales poorly because as the dataset size (N) increases, the time required to find the k -nearest neighbors grows linearly ($O(N)$), making predictions unacceptably slow and memory-intensive for millions of points.
3. KNN is rarely used in production because its high latency during prediction makes it too slow for real-time applications where users expect instant results. Additionally, it struggles with the Curse of Dimensionality, where distance metrics become less effective as the number of features increases, and the high cost of keeping massive datasets in active memory makes it less economical than parametric alternatives.

Conclusion [1 Mark]

Write a brief summary of what you learned from this lab, focusing on the importance of understanding algorithmic foundations before using high-level libraries.

Implementing these algorithms from scratch reveals that understanding mathematical foundations, like learning rates and vectorization, is essential for building efficient models and debugging the failure modes that high-level libraries often hide.

Assessment Rubrics

SE306L: Machine Learning Lab

Method: Lab reports and instructor observation during lab sessions

Outcome Assessed:

- a. Ability to conduct experiments, as well as to analyze and interpret data (P).
- b. Ability to use the techniques, skills, and modern engineering tools necessary for engineering practice (P).

Performance	Exceeds expectation (4-5)	Meets expectation (3-2)	Does not meet expectation (1)	Marks
1. Realization of Experiment [a, b]	Selects relevant equipment to the experiment, develops setup diagrams of equipment connections or wiring.	Needs guidance to select relevant equipment to the experiment and to develop equipment connection or wiring diagrams.	Incapable of selecting relevant equipment to conduct the experiment, equipment connection or wiring diagrams are	
2. Conducting Experiment [a, b]	Does proper calibration of equipment, carefully examines equipment moving parts, and ensures smooth operation and process.	Calibrates equipment, examines equipment moving parts, and operates the equipment with minor error.	Unable to calibrate appropriate equipment, and equipment operation is substantially wrong.	
3. Laboratory Safety Rules [a]	Respectfully and carefully observes safety rules and procedures	Observes safety rules and procedures with minor deviation.	Disregards safety rules and procedures.	
6. Data Collection [a]	Plans data collection to achieve experimental objectives, and conducts an orderly and a complete data collection.	Plans data collection to achieve experimental objectives, and collects complete data with minor error.	Does not know how to plan data collection to achieve experimental goals; data collected is incomplete and contain errors.	
7. Data Analysis [a]	Accurately conducts simple computations and statistical analysis using collected data; correlates experimental results to known theoretical values; accounts for measurement errors and parameters that affect experimental results.	Conducts simple computations and statistical analysis using collected data with minor error; reasonably correlates experimental results to known theoretical values; attempts to account for measurement errors and parameters that affect experimental results.	Unable to conduct simple statistical analysis on collected data; no attempt to correlate experimental results with known theoretical values; incapable of explaining measurement errors or parameters that affect the experimental results.	
8. Computer Use [a]	Uses computer to collect and analyze data effectively.	Uses computer to collect and analyze data with minor error.	Does not know how to use computer to collect and analyze data.	
Total				

Faculty:

Name: _____

Signature: _____

Date: _____